

SWE 4739

Embedded Software Development

Lecture 5

Design Patterns

Sabrina Islam

Lecturer, CSE, IUT

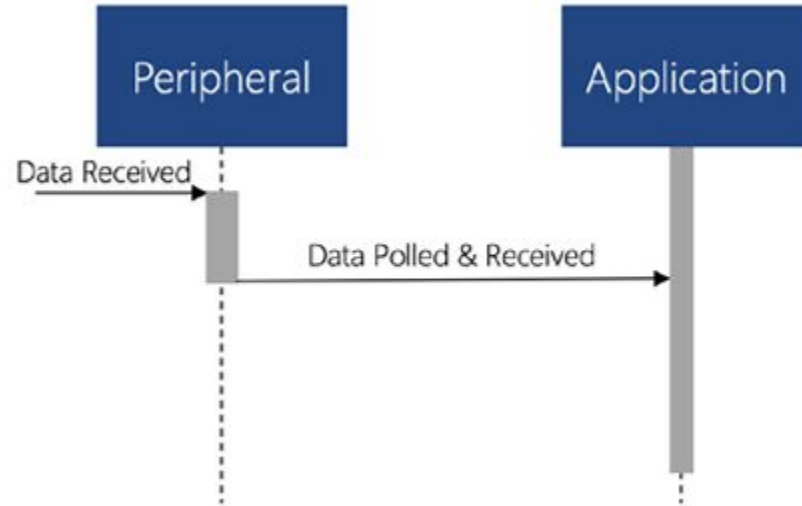
Contact: +8801832239897

E-mail: sabrinaislam22@iut-dhaka.edu

Managing Peripheral Data

- Major design decision: “How to get the data from the peripheral to the application?”
- Different design mechanisms:
 - Polling
 - Interrupts
 - Direct memory access (DMA)

Peripheral Polling



Peripheral Polling

- Types of Polling
 - Blocking Polling
 - CPU continuously waits for data (e.g., while loop)
 - Low latency and consistent timing
 - High CPU wastage
 - Non-Blocking Polling
 - CPU performs other tasks while checking periodically
 - Better CPU utilization
 - Increased latency if data arrives during other tasks

Peripheral Polling

Advantages

- Simple to implement
- No interrupt controller configuration required
- Easy to debug

Disadvantages

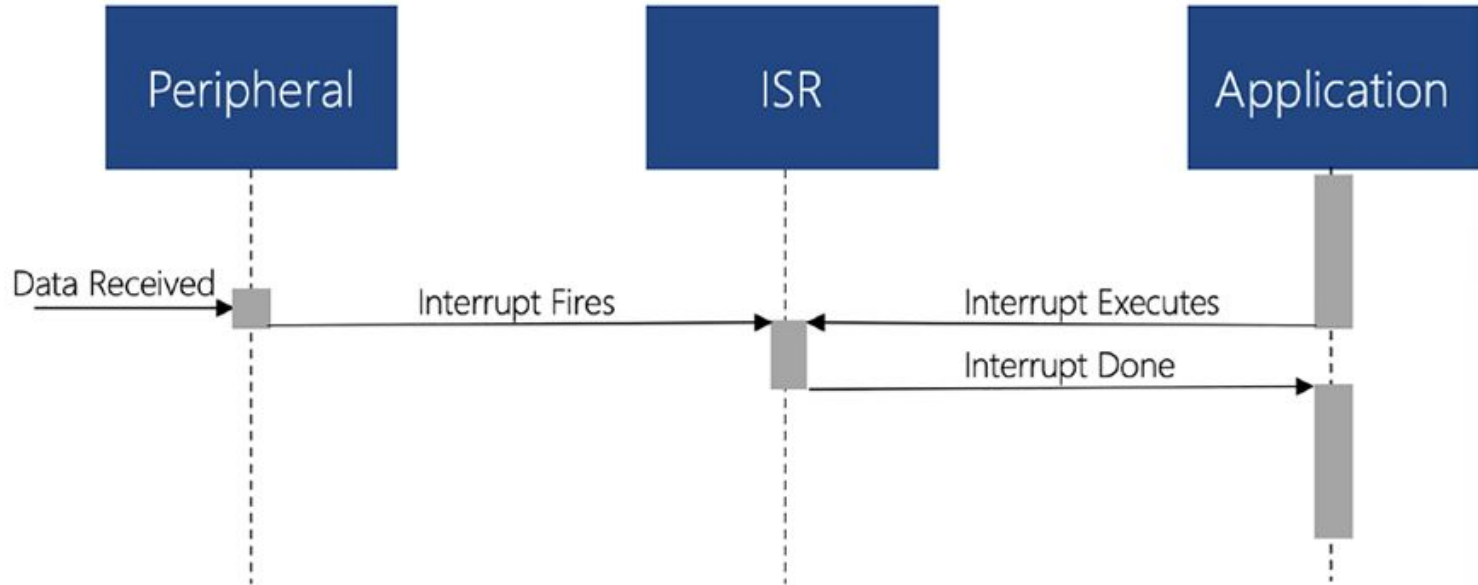
- Wastes CPU processing cycles
- Inefficient for low-power or resource-constrained systems
- Can introduce latency (delay in response)
- Can cause jitter (inconsistent response timing)

Peripheral Interrupts

How Interrupts Work

- Peripheral detects an event
- Interrupt signal is generated
- CPU pauses current task
- ISR executes to handle the event
- CPU resumes normal execution

Peripheral Interrupts



Peripheral Interrupts

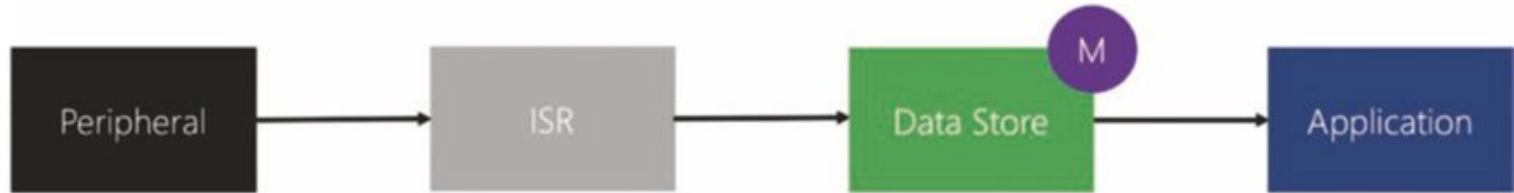
Advantages

- No CPU cycle wastage (no constant checking)
- Efficient for real-time systems
- Deterministic latency (predictable response time)
- Reduced jitter compared to polling

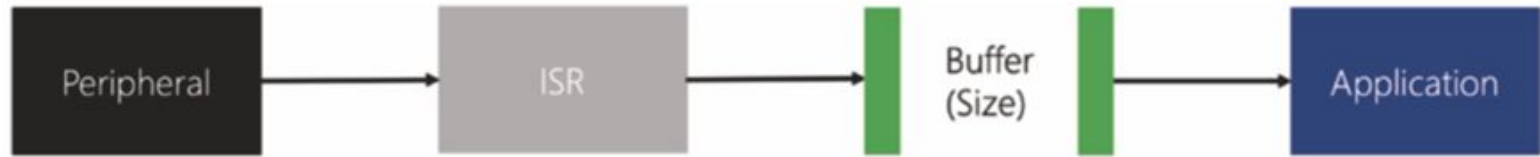
Interrupt Design Patterns

- Linear data store
- Ping-pong buffers
- Circular buffers
- Circular buffer with semaphores
- Circular buffer with event flags
- Message queues

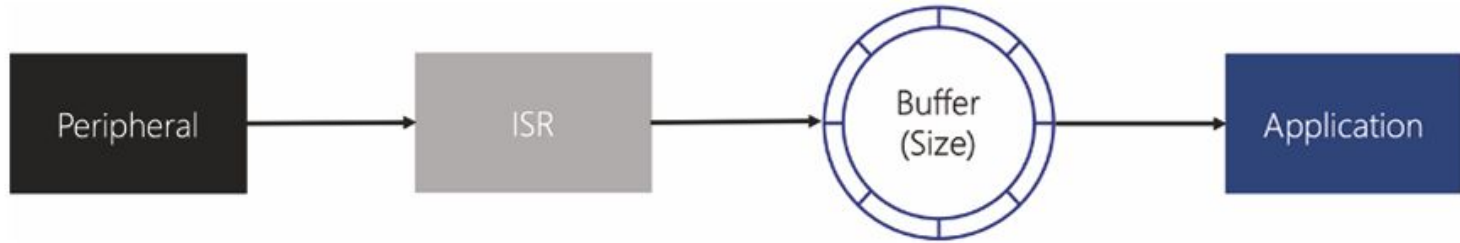
Linear Data Store Design Pattern



Ping-Pong Buffer Design Pattern

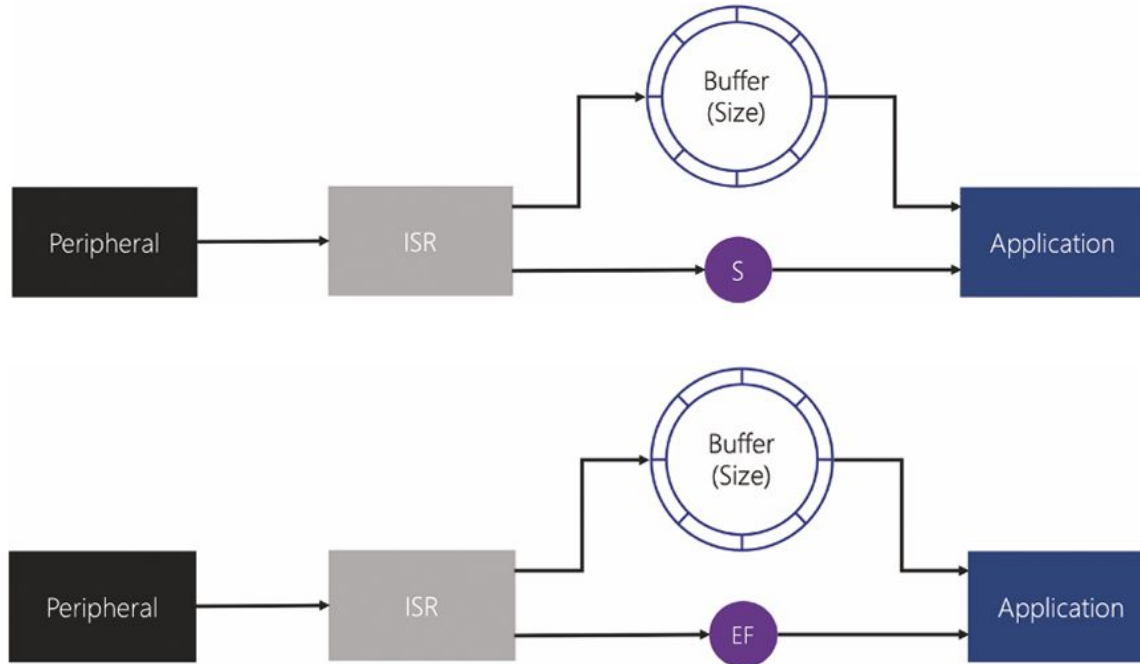


Circular Buffer Design Pattern

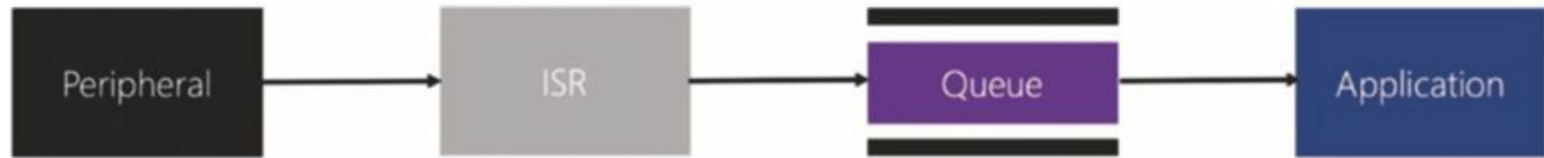


S(1)...2....3....4...5.....S(8)

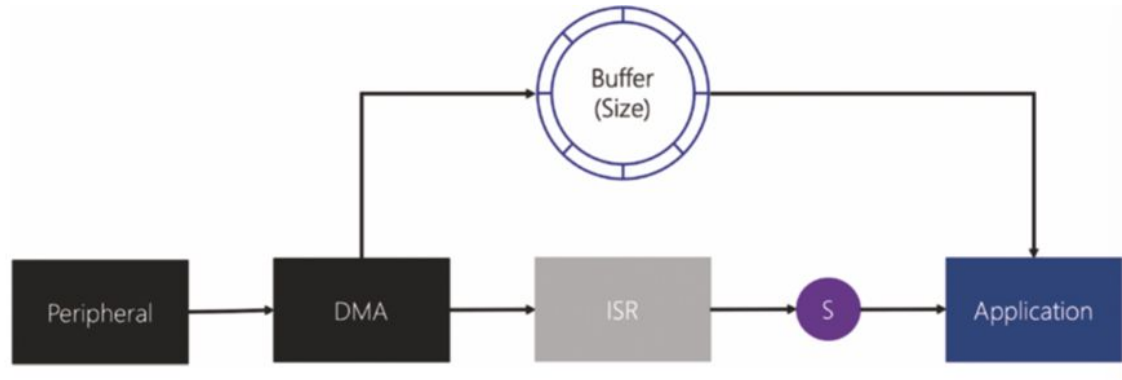
Circular Buffer with Notification Design Pattern



Message Queue Design Pattern



Direct Memory Access (DMA)



Resource Synchronization

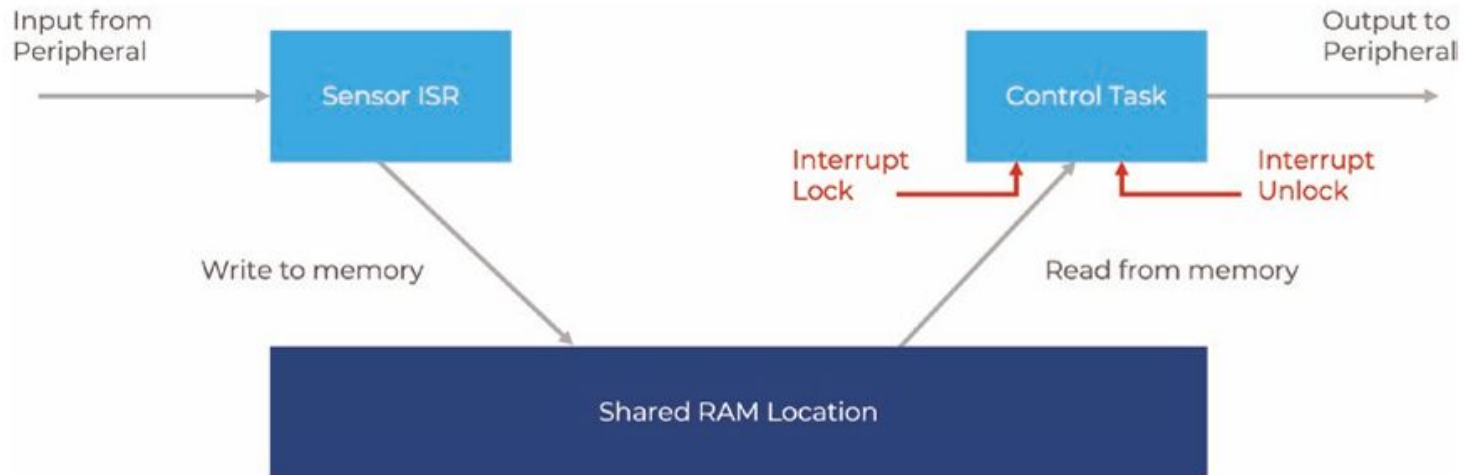
- Ensures safe access to shared resources (e.g., memory)
- Prevents race conditions and data corruption
- Occurs when multiple tasks or interrupts access the same resource

Resource Synchronization

- Ensures safe access to shared resources (e.g., memory)
- Prevents race conditions and data corruption
- Occurs when multiple tasks or interrupts access the same resource
- Types:
 - Interrupt Locking
 - Preemption Locking
 - Mutex Lock

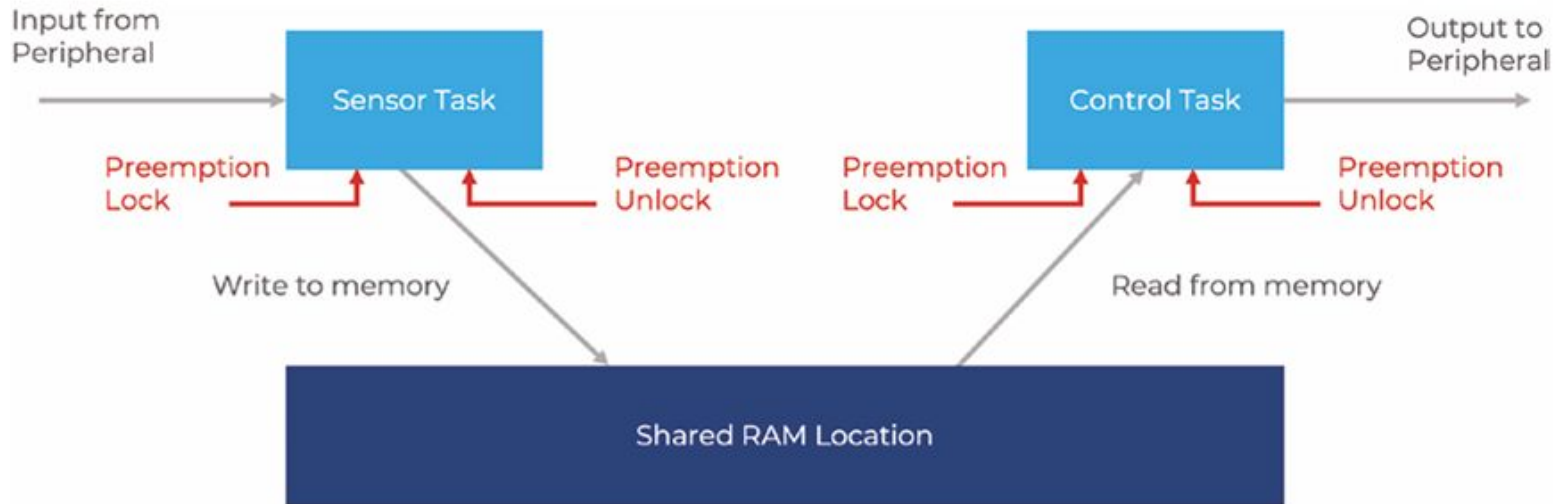
Resource Synchronization

Interrupt Locking



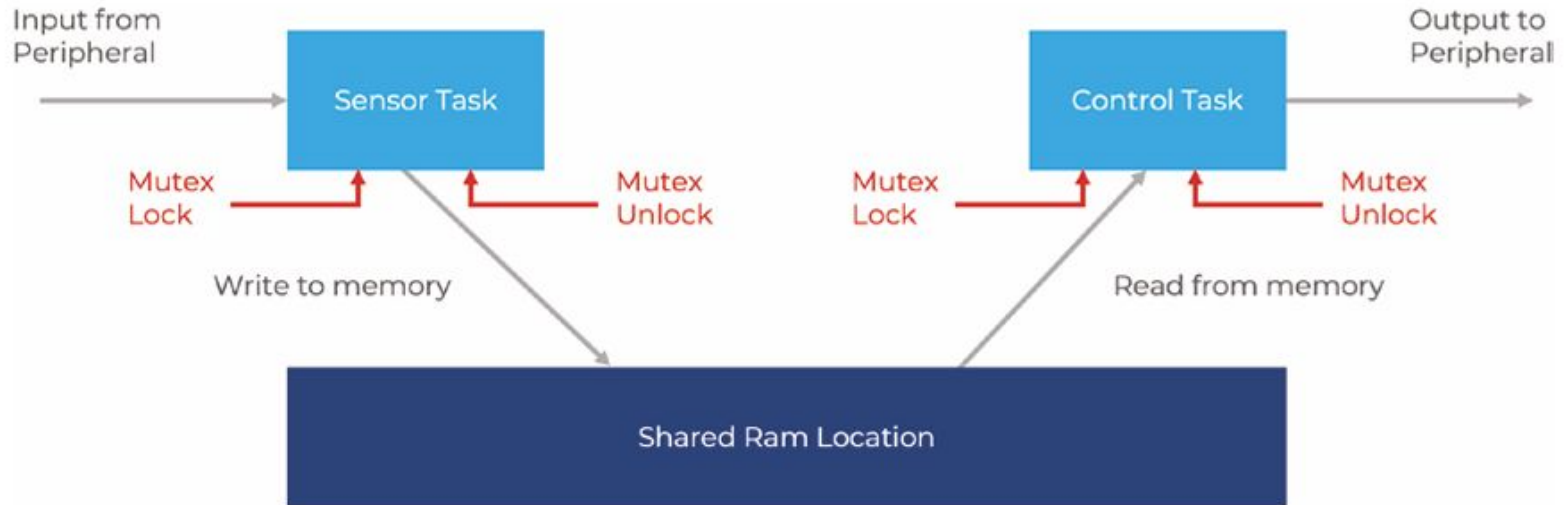
Resource Synchronization

Preemption Locking



Resource Synchronization

Mutex Lock

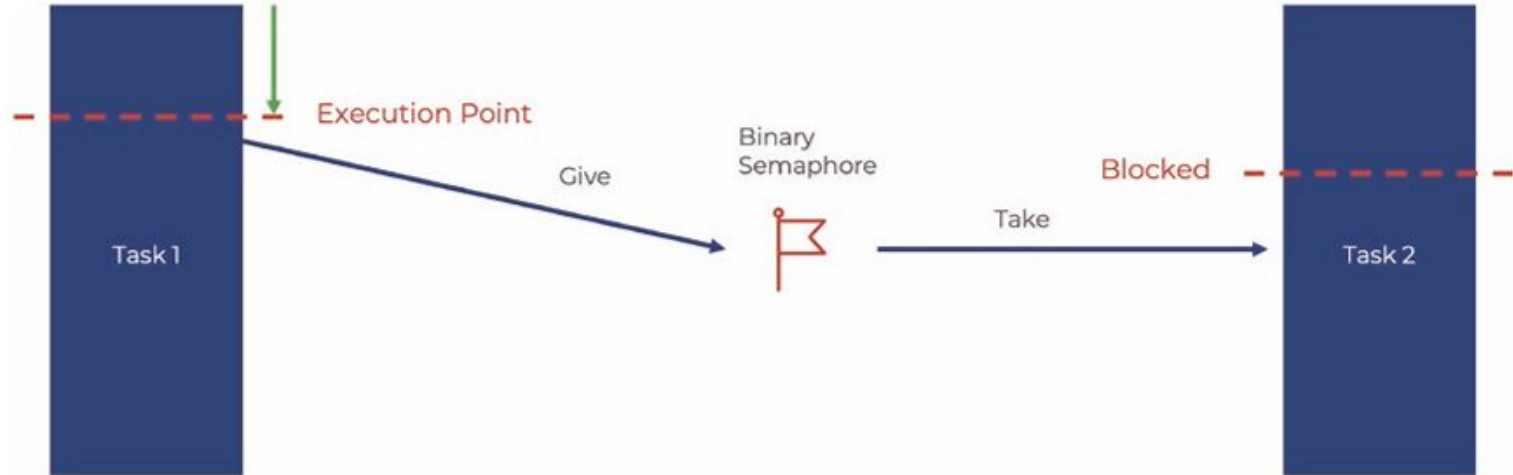


Activity Synchronization

- Coordinates execution between tasks (or interrupts)
- Ensures tasks act on fresh data, not stale values
- Types:
 - Unilateral Rendezvous (Task → Task)
 - Unilateral Rendezvous (Interrupt → Task)
 - Bilateral Rendezvous

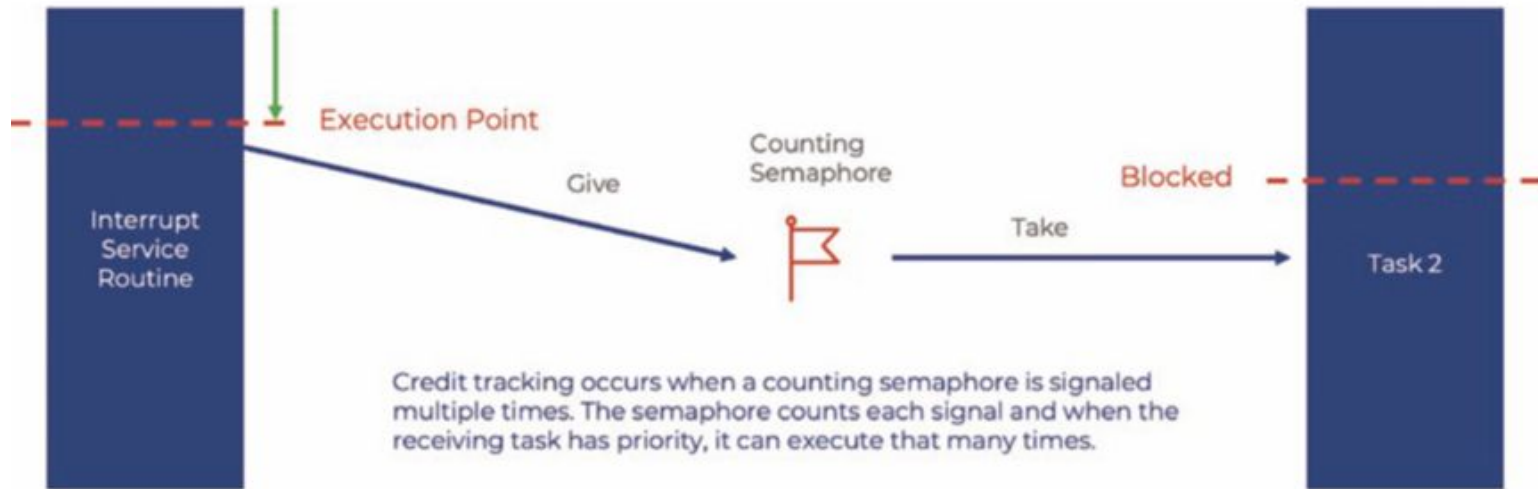
Activity Synchronization

Unilateral Rendezvous (Task → Task)



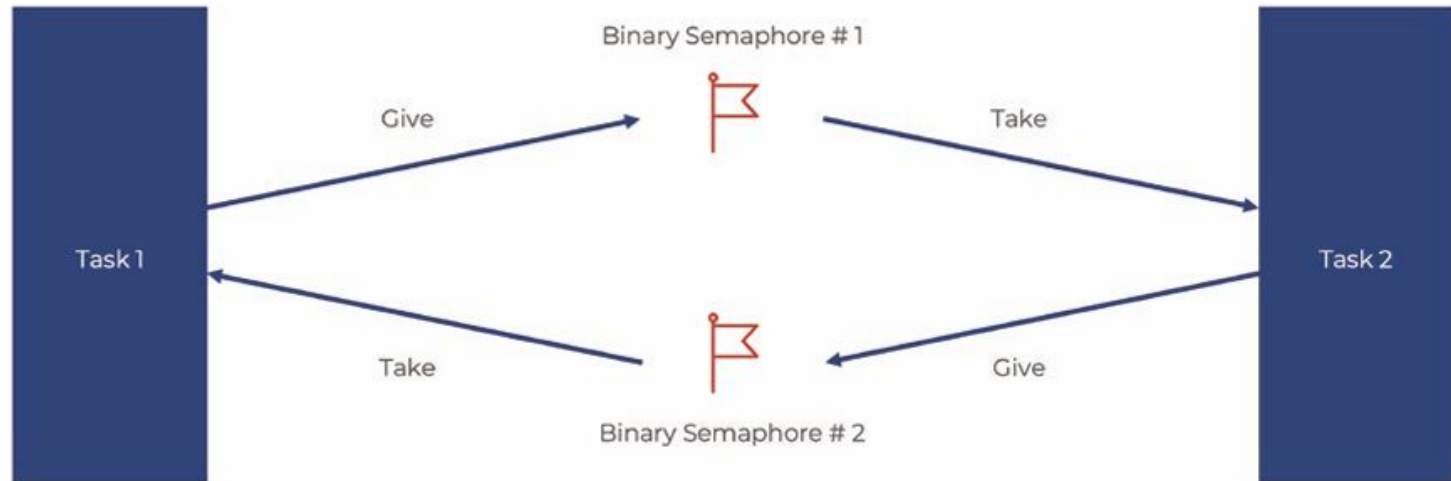
Activity Synchronization

Unilateral Rendezvous (Interrupt → Task)



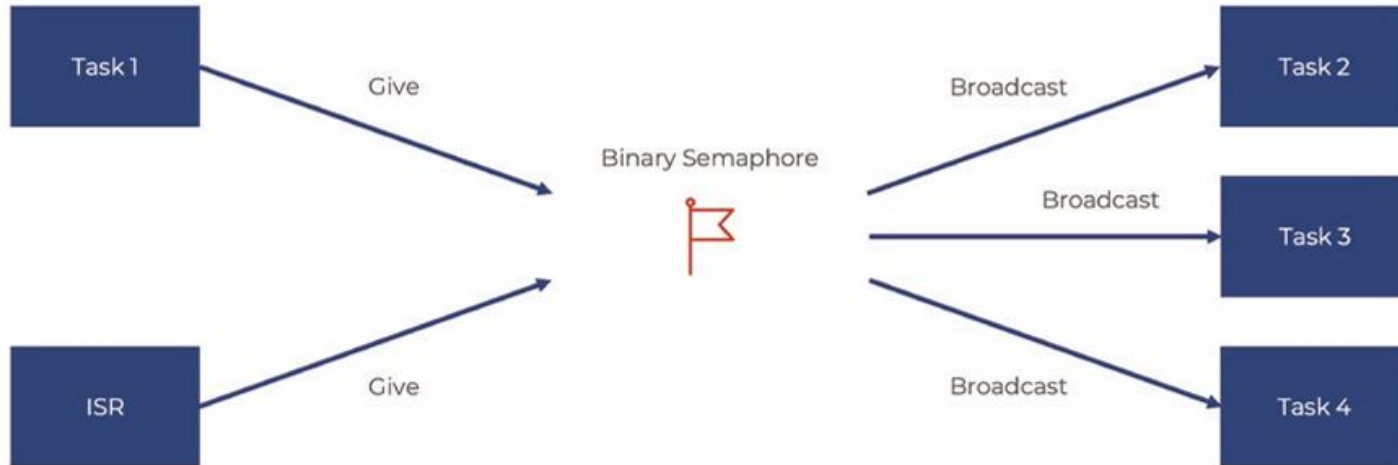
Activity Synchronization

Bilateral Rendezvous



Synchronizing Multiple Tasks

- One event signals multiple tasks simultaneously
- Tasks unblock and execute based on priority
- Broadcast design pattern



Reference:

1. Embedded Software Design: A Practical Approach to Architecture, Processes, and Coding Technique – Jacob Beningo

Thank You!